

Indexing and Hashing

An index for a file in a database system works in much the same way as the index in this textbook. Database-system indices play the same role as book indices in libraries.

For example, to retrieve a student record given an ID, the database system would look up an index to find on which disk block the corresponding record resides, and then fetch the disk block, to get the appropriate student record.

Keeping a sorted list of students' ID would not work well on very large databases with thousands of students, since the index would itself be very big; further, even though keeping the index sorted reduces the search time, finding a student can still be rather time-consuming.

Instead, more sophisticated indexing techniques may be used.

There are two basic kinds of indices:

- **Ordered indices.** Based on a sorted ordering of the values.
- **Hash indices.** Based on a uniform distribution of values across a range of buckets. The bucket to which a value is assigned is determined by a function, called a hash function.

We shall consider several techniques for both ordered indexing and hashing. Each technique must be evaluated on the basis of these factors:

- **Access types:** The types of access that are supported efficiently. Access types can include finding records with a specified attribute value and finding records whose attribute values fall in a specified range.
- **Access time:** The time it takes to find a particular data item, or set of items, using the technique in question.
- **Insertion time:** The time it takes to insert a new data item. This value includes the time it takes to find the correct place to insert the new data item, as well as the time it takes to update the index structure.
- **Deletion time:** The time it takes to delete a data item. This value includes the time it takes to find the item to be deleted, as well as the time it takes to update the index structure.
- **Space overhead:** The additional space occupied by an index structure. Provided that the amount of additional space is moderate, it is usually worthwhile to sacrifice the space to achieve improved performance.

Screen clipping taken: 07-06-2026 09:14

We often want to have more than one index for a file. For example, we may wish to search for a book by author, by subject, or by title.

An attribute or set of attributes used to look up records in a file is called a **search key**. Note that this definition of key differs from that used in primary key, candidate key, and super key.

11.2 Ordered Indices

To gain fast random access to records in a file, we can use an index structure.

Each index structure is associated with a particular search key.

Just like the index of a book or a library catalog,

an ordered index stores the values of the search keys in sorted order, and associates with each search key the records that contain it.

The records in the indexed file may themselves be stored in some sorted order,

A file may have several indices, on different search keys.

If the file containing the records is sequentially ordered, a clustering index is an index whose search key also defines the sequential order of the file.

Clustering indices are also called primary indices.

Indices whose search key specifies an order different from the sequential order of the file are called **non clustering** indices, or secondary indices.

The terms "clustered" and "non clustered" are often used in place of "clustering" and "non clustering."

all files are ordered sequentially on some search key. Such files, with a clustering index on the search key, are called **index-sequential** files.

Figure 11.1 shows a sequential file of *instructor* records taken from our university example. In the example of Figure 11.1, the records are stored in sorted order of instructor *ID*, which is used as the search key.

Screen clipping taken: 07-06-2026 09:56

10101	Srinivasan	Comp. Sci.	65000	
12121	W...	Eng...	00000	

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

Figure 11.1 Sequential file for *instructor* records.

Screen clipping taken: 07-06-2026 09:56

11.2.1 Dense and Sparse Indices

An **index entry**, or **index record**, consists of a search-key value and pointers to one or more records with that value as their search-key value. The pointer to a record consists of the identifier of a disk block and an offset within the disk block to identify the record within the block.

There are two types of ordered indices that we can use:

Screen clipping taken: 07-06-2026 09:57

Dense index:

In a dense index, an index entry appears for every search-key value in the file. In a dense clustering index, the index record contains the search-key value and a pointer to the first data record with that search-key value.

The rest of the records with the same search-key value would be stored sequentially after the first record, since, because the index is a clustering one, records are sorted on the same search key.

In a dense non clustering index, the index must store a list of pointers to all records with the same search-key value.

Sparse index: In a sparse index, an index entry appears for only some of the search-key values. Sparse indices can be used only if the relation is stored in sorted order of the search key, that is, if the index is a clustering index.

To locate a record, we find the index entry with the largest search-key value that is less than or equal to the search-key value for which we are looking. We start at the record pointed to by that index entry, and follow the pointers in the file until we find the desired record.

Multilevel Indices:

Binary search can be used on the index file to locate an entry, but the search still has a large cost. If the index would occupy b blocks, binary search requires as many as $\lceil \log_2(b) \rceil$ blocks to be read. ($\lceil x \rceil$ denotes the least integer that is greater than or equal to x ; that is, we round upward.)

Since there are many demands on main memory, it may not be possible to reserve that much main memory just for this particular outer index. In such a case, we can create yet another level of index. Indeed, we can repeat this process as many times as necessary

Indices with two or more levels are called multilevel indices. Searching for records with a multilevel index requires significantly fewer I/O operations than does searching for records by binary search

Multilevel indices are closely related to tree structures, such as the binary trees used for in-memory indexing.

B⁺-Tree Index Files

The main disadvantage of the index-sequential file organization is that performance degrades as the file grows, both for index lookups and for sequential scans through the data

Although this degradation can be remedied by reorganization of the file, frequent reorganizations are undesirable.

- The B⁺-tree index structure is the most widely used of several index structures that maintain their efficiency despite insertion and deletion of data.
- A B⁺-tree index takes the form of a balanced tree in which every path from the root of the tree to a leaf of the tree is of the same length.
- Each non leaf node in the tree has between $\lceil n/2 \rceil$ and n children, where n is fixed for a particular tree.
- We shall see that the B⁺-tree structure imposes performance overhead on insertion and deletion, and adds space overhead.
- The overhead is acceptable even for frequently modified files, since the cost of file reorganization is avoided.
- Furthermore, since nodes may be as much as half empty (if they have the minimum number of children), there is some wasted space. This space overhead, too, is acceptable given the performance benefits of the B⁺-tree structure.

P_1	K_1	P_2	$\dots$	P_{n-1}	K_{n-1}	P_n
-------	-------	-------	---------	-----------	-----------	-------

Figure 11.7 Typical node of a B⁺-tree.

Screen clipping taken: 07-06-2026 15:12

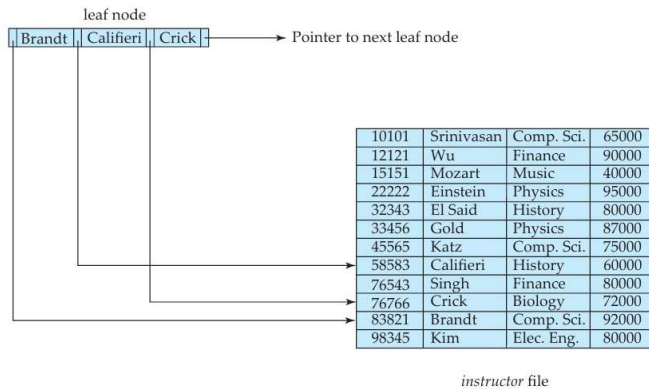


Figure 11.8 A leaf node for *instructor* B⁺-tree index ($n = 4$).

Screen clipping taken: 07-06-2026 15:12

11.4.5 B-Tree Index Files

B-tree indices are similar to B⁺-tree indices. The primary distinction between the two approaches is that a B-tree eliminates the redundant storage of search-key values.

A B-tree allows search-key values to appear only once (if they are unique), unlike a B⁺-tree, where a value may appear in a nonleaf node, in addition to appearing in a leaf node.

Since search keys are not repeated in the B-tree, we may be able to store the index in fewer tree nodes than in the corresponding B⁺-tree index.

since search keys that appear in nonleaf nodes appear nowhere else in the B-tree

we are forced to include an additional pointer field for each search key in a nonleaf node.

These additional pointers point to either file records or buckets for the associated search key.

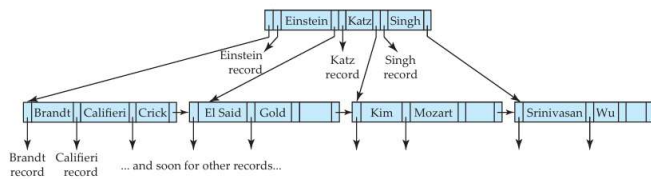


Figure 11.21 B-tree equivalent of B⁺-tree in Figure 11.13.

Screen clipping taken: 12-06-2026 05:59

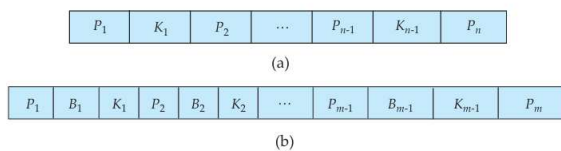


Figure 11.22 Typical nodes of a B-tree. (a) Leaf node. (b) Nonleaf node.

Screen clipping taken: 12-06-2026 06:07

Multiple-Key Access

Screen clipping taken: 07-06-2026 16:48

Until now, we have assumed implicitly that only one index on one attribute is used to process a query on a relation

However, for certain types of queries, it is advantageous to use multiple indices if they exist, or to use an index built on a multi attribute search key.

11.5.1 Using Multiple Single-Key Indices

Assume that the *instructor* file has two indices: one for *dept_name* and one for *salary*. Consider the following query: “Find all instructors in the Finance department with salary equal to \$80,000.” We write

```
select ID
from instructor
where dept_name = 'Finance' and salary = 80000;
```

There are three strategies possible for processing this query:

1. Use the index on *dept_name* to find all records pertaining to the Finance department. Examine each such record to see whether *salary* = 80000.
2. Use the index on *salary* to find all records pertaining to instructors with salary of \$80,000. Examine each such record to see whether the department name is “Finance”.
3. Use the index on *dept_name* to find pointers to all records pertaining to the Finance department. Also, use the index on *salary* to find pointers to all records pertaining to instructors with a salary of \$80,000. Take the intersection of these two sets of pointers. Those pointers that are in the intersection point to records pertaining to instructors of the Finance department and with salary of \$80,000.

Screen clipping taken: 07-06-2026 17:09

The third strategy is the only one of the three that takes advantage of the existence of multiple indices. However, even this strategy may be a poor choice if all of the following hold:

- There are many records pertaining to the Finance department.
- There are many records pertaining to instructors with a salary of \$80,000.
- There are only a few records pertaining to *both* the Finance department and instructors with a salary of \$80,000.

If these conditions hold, we must scan a large number of pointers to produce a small result. An index structure called a “bitmap index” can in some cases greatly speed up the intersection operation used in the third strategy. Bitmap indices are outlined in Section 11.9.

Screen clipping taken: 07-06-2026 17:10

11.5.2 Indices on Multiple Keys

An alternative strategy for this case is to create and use an index on a composite search key (*dept_name, salary*)—that is, the search key consisting of the department name concatenated with the instructor salary.

We can use an ordered (B⁺-tree) index on the above composite search key to answer efficiently queries of the form

```
select ID
from instructor
where dept_name = 'Finance' and salary = 80000;
```

Screen clipping taken: 07-06-2026 17:10

```
select ID
from instructor
where dept_name = 'Finance' and salary < 80000;
```

Screen clipping taken: 07-06-2026 17:11

Static Hashing

Screen clipping taken: 07-06-2026 17:14

- One disadvantage of sequential file organization is that we must access an index structure to locate data, or must use binary search, and that results in more I/O operations.
- File organizations based on the technique of hashing allow us to avoid accessing an index structure. Hashing also provides a way of constructing indices.
- bucket to denote a unit of storage that can store one or more records. A bucket is typically a disk block, but could be chosen to be smaller or larger than a disk block.
- Formally, let K denote the set of all search-key values, and let B denote the set of all bucket addresses. A hash function h is a function from K to B . Let h denote a hash function.
- To insert a record with search key K_i , we compute $h(K_i)$, which gives the address of the bucket for that record. Assume for now that there is space in the bucket to store the record. Then, the record is stored in that bucket.
- To perform a lookup on a search-key value K_i , we simply compute $h(K_i)$, then search the bucket with that address. Suppose that two search keys, K_5 and K_7 , have the same hash value; that is, $h(K_5) = h(K_7)$. If we perform a lookup on K_5 , the bucket $h(K_5)$ contains records with search-key values K_5 and records with

search-key values K_7 . Thus, we have to check the search-key value of every record in the bucket to verify that the record is one that we want.

- Deletion is equally straightforward. If the search-key value of the record to be deleted is K_i , we compute $h(K_i)$, then search the corresponding bucket for that record, and delete the record from the bucket.
- Hashing can be used for two different purposes. In a hash file organization, we obtain the address of the disk block containing a desired record directly by computing a function on the search-key value of the record.
- In a hash index organization we organize the search keys, with their associated pointers, into a hash file structure.

Dynamic Hashing

Several **dynamic hashing** techniques allow the hash function to be modified dynamically to accommodate the **growth or shrinkage** of the database. In this section we describe one form of dynamic hashing, called **extendable hashing**.

Extendable hashing copes with changes in database size by splitting and coalescing buckets as the database grows and shrinks. As a result, space efficiency is retained. Moreover, since the reorganization is performed on only one bucket at a time, the resulting performance overhead is acceptably low.

However, this hash function generates values over a relatively large range—namely, b -bit binary integers. A typical value for b is 32. We do not create a bucket for each hash value

we create buckets on demand, as records are inserted into the file. We do not use the entire b bits of the hash value initially. At any point, we use i bits, where $0 \leq i \leq b$. These i bits are used as an offset into an additional table of bucket addresses. The value of i grows and shrinks with the size of the database.

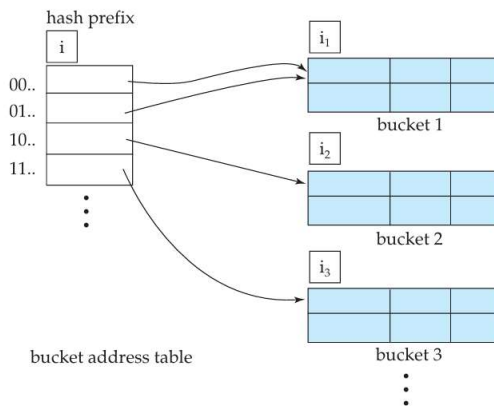


Figure 11.26 General extendable hash structure.

Screen clipping taken: 07-06-2026 23:20

Static Hashing versus Dynamic Hashing:

The main advantage of extendable hashing is that :
performance does not degrade as the file grows.
Furthermore, there is minimal space overhead.

Although the bucket address table incurs additional overhead, it contains one pointer for each hash value for the current prefix length. This table is thus small. The main space saving of extendable hashing over other forms of hashing is that no buckets need to be reserved for future growth; rather, buckets can be allocated dynamically.

A disadvantage of extendable hashing is that lookup involves an additional level of indirection, since the system must access the bucket address table before accessing the bucket itself.

Comparison of Ordered Indexing and Hashing

Screen clipping taken: 07-06-2026 23:31

Bitmap Indices

Screen clipping taken: 12-06-2026 05:23

Bitmap indices are a specialized type of index designed for easy querying on multiple keys, although each bitmap index is built on a single key. For bitmap indices to be used, records in a relation must be numbered sequentially, starting from, say, 0. Given a number n , it must be easy to retrieve the record numbered n

Bitmap Index Structure:

A bitmap is simply an array of bits. In its simplest form, a bitmap index on the attribute A of relation r consists of one bitmap for each value that A can take. Each bitmap has as many bits as the number of records in the relation. The i th bit of the bitmap for value v_j is set to 1 if the record numbered i has the value v_j for attribute A . All other bits of the bitmap are set to 0.

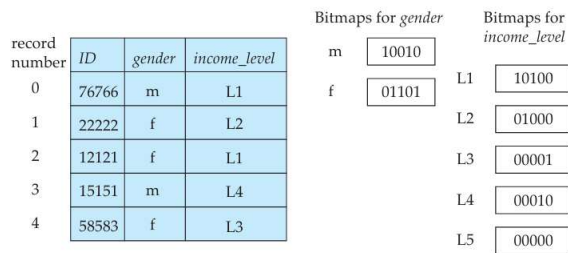


Figure 11.35 Bitmap indices on relation *instructor.info*.

Screen clipping taken: 12-06-2026 05:40

In fact, bitmap indices are useful for selections mainly when there are selections on multiple keys. Suppose we create a bitmap index on attribute *income_level*, which we described earlier, in addition to the bitmap index on *gender*.

Consider now a query that selects women with income in the range \$10,000 to \$19,999. This query can be expressed as

```
select *
from r
where gender = 'f' and income_level = 'L2';
```

Screen clipping taken: 12-06-2026 05:42

TRANSACTION MANAGEMENT

The term transaction refers to a collection of operations that form a single logical unit of work.

For instance, transfer of money from one account to another is a transaction consisting of two updates, one to each account.

It is important that either all actions of a transaction be executed completely, or, in case of some failure, partial effects of each incomplete transaction be undone. This property is called **atomicity**.

once a transaction is successfully executed, its effects must persist in the database—a system failure should not result in the database forgetting about a transaction that successfully completed. This property is called **durability**.

database systems must provide mechanisms to isolate transactions from the effects of other concurrently executing transactions. This property is called **isolation**.

the transaction-management component of a database system allows application developers to focus on the implementation of individual transactions, ignoring the issues of concurrency and fault tolerance.

Collections of operations that form a single logical unit of work are called **transactions**

Transaction Concept

Screen clipping taken: 12-06-2026 06:42

A **transaction** is a **unit** of program execution that accesses and possibly updates various data items. Usually, a transaction is initiated by a user program written in a high-level data-manipulation language (typically SQL), or programming language (for example, C++, or Java), with embedded database accesses in JDBC or ODBC. A transaction is delimited by statements (or function calls) of the form **begin transaction** and **end transaction**. The transaction consists of all operations executed between the **begin transaction** and **end transaction**.

This collection of steps must appear to the user as a single, indivisible unit. Since a transaction is indivisible, it either executes in its entirety or not at all. Thus, if a transaction begins to execute but fails for whatever reason, any changes to the database that the transaction may have made must be undone. This requirement holds regardless of whether the transaction itself failed, the operating system crashed, or the computer itself stopped operating.

This “all-or-none” property is referred to as **atomicity**.

the database system must take special actions to ensure that transactions operate properly without interference from concurrently executing database statements. This property is referred to as **isolation**.

Even if the system ensures correct execution of a transaction, this serves little purpose if the system subsequently crashes and, as a result, the system “forgets” about the transaction. Thus, a transaction’s actions must persist across crashes. This property is referred to as **durability**.

- **Atomicity.** Either all operations of the transaction are reflected properly in the database, or none are.
- **Consistency.** Execution of a transaction in isolation (that is, with no other transaction executing concurrently) preserves the consistency of the database.
- **Isolation.** Even though multiple transactions may execute concurrently, the system guarantees that, for every pair of transactions T_i and T_j , it appears to T_i that either T_j finished execution before T_i started or T_j started execution after T_i finished. Thus, each transaction is unaware of other transactions executing concurrently in the system.
- **Durability.** After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

Screen clipping taken: 12-06-2026 06:57

These properties are often called the **ACID properties**; the acronym is derived from the first letter of each of the four properties.

A Simple Transaction Model

Screen clipping taken: 12-06-2026 07:11

when data are moved from disk to main memory and from main memory to disk. In doing this, we ignore SQL insert and delete operations, We shall illustrate the transaction concept using a simple bank application consisting of several accounts and a set of transactions that access and update those accounts.

Transactions access data using two operations

- **read(X),** which transfers the data item X from the database to a variable, also called X , in a buffer in main memory belonging to the transaction that executed the **read** operation.
- **write(X),** which transfers the value in the variable X in the main-memory buffer of the transaction that executed the **write** to the data item X in the database.

Screen clipping taken: 12-06-2026 07:13

It is important to know if a change to a data item appears only in main memory or if it has been written to the database on disk.

In a real database system, the write operation does not necessarily result in the immediate update of the data on the disk; the write operation may be temporarily stored elsewhere and executed on the disk later.

```
Ti: read(A);
   A := A - 50;
   write(A);
   read(B);
   B := B + 50;
   write(B).
```

Screen clipping taken: 12-06-2026 07:15

- **Consistency:** The consistency requirement here is that the sum of A and B be unchanged by the execution of the transaction. Without the consistency requirement, money could be created or destroyed by the transaction! It can be verified easily that, if the database is consistent before an execution of the transaction, the database remains consistent after the execution of the transaction.

Ensuring consistency for an individual transaction is the responsibility of the application programmer who codes the transaction. This task may be facilitated by automatic testing of integrity constraints, as we discussed in Section 4.4.

Screen clipping taken: 12-06-2026 07:17

- **Atomicity:**

Screen clipping taken: 12-06-2026 07:19

The basic idea behind ensuring atomicity is this: The database system keeps track (on disk) of the old values of any data on which a transaction performs a write. This information is written to a file called the log. If the transaction does not complete its execution, the database system restores the old values from the log to make it appear as though the transaction never executed.

Ensuring atomicity is the responsibility of the database system; specifically, it is handled by a component of the database called the recovery system

- **Durability:** Once the execution of the transaction completes successfully, and the user who initiated the transaction has been notified that the transfer of

14.2 A Simple Transaction Model 631

funds has taken place, it must be the case that no system failure can result in a loss of data corresponding to this transfer of funds. The durability property guarantees that, once a transaction completes successfully, all the updates that it carried out on the database persist, even if there is a system failure after the transaction completes execution.

Screen clipping taken: 12-06-2026 07:21

1. The updates carried out by the transaction have been written to disk before the transaction completes.
2. Information about the updates carried out by the transaction and written to disk is sufficient to enable the database to reconstruct the updates when the database system is restarted after the failure.

Screen clipping taken: 12-06-2026 07:22

- **Isolation:** Even if the consistency and atomicity properties are ensured for each transaction, if several transactions are executed concurrently, their operations may interleave in some undesirable way, resulting in an inconsistent state.

Screen clipping taken: 12-06-2026 07:25

Transaction Atomicity and Durability

Screen clipping taken: 12-06-2026 07:33

a transaction may not always complete its execution successfully. Such a transaction is termed **aborted**. If we are to ensure the **atomicity property**, an aborted transaction must have no effect on the state of the database. Thus, any changes that the aborted transaction made to the database must be undone. Once the changes caused by an aborted transaction have been undone, we say that the transaction has been **undone**. It is part of the responsibility of the **recovery scheme** to manage transaction **aborts**.

This is done typically by maintaining a log. Each database modification made by a transaction is first recorded in the log. We record the identifier of the transaction performing the modification, the identifier of the data item being modified, and both the old value (prior to modification) and the new value (after modification) of the data item.

Maintaining a log provides the possibility of redoing a modification to ensure atomicity and durability as well as the possibility of undoing a modification to ensure atomicity in case of a failure during transaction execution.

A transaction that completes its execution successfully is said to be **committed**. A committed transaction that has performed updates transforms the database into a new consistent state, which must persist even if there is a system failure.

- **Active**, the initial state; the transaction stays in this state while it is executing.
- **Partially committed**, after the final statement has been executed.
- **Failed**, after the discovery that normal execution can no longer proceed.
- **Aborted**, after the transaction has been rolled back and the database has been restored to its state prior to the start of the transaction.
- **Committed**, after successful completion.

The state diagram corresponding to a transaction appears in Figure 14.1. We say that a transaction has committed only if it has entered the committed state.

Screen clipping taken: 12-06-2026 07:39

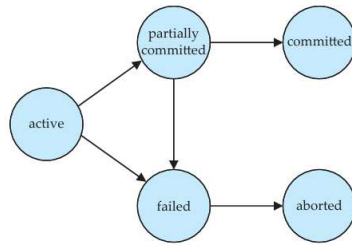


Figure 14.1 State diagram of a transaction.

Screen clipping taken: 12-06-2026 07:40

Similarly, we say that a transaction has aborted only if it has entered the aborted state. A transaction is said to have terminated if it has either committed or aborted.

1. A transaction starts in the active state.
2. When it finishes its final statement, it enters the partially committed state.
3. At this point, the transaction has completed its execution, but it is still possible that it may have to be aborted, since the actual output may still be temporarily residing in main memory, and thus a hardware failure may preclude its successful completion.
4. The database system then writes out enough information to disk that, even in the event of a failure, the updates performed by the transaction can be re-created when the system restarts after the failure. When the last of this information is written out, the transaction enters the committed state.
5. A transaction enters the failed state after the system determines that the transaction can no longer proceed with its normal execution (for example, because of hardware or logical errors).
6. Such a transaction must be rolled back. Then, it enters the aborted state. At this point, the system has two options:
 - It can **restart** the transaction, but only if the transaction was aborted as a result of some hardware or software error that was not created through the internal logic of the transaction. A restarted transaction is considered to be a new transaction.
 - It can **kill** the transaction. It usually does so because of some internal logical error that can be corrected only by rewriting the application program, or because the input was bad, or because the desired data were not found in the database.

Screen clipping taken: 12-06-2026 07:45

Transaction Isolation

Screen clipping taken: 12-06-2026 07:46

Ensuring consistency in spite of concurrent execution of transactions requires extra work; it is far easier to insist that transactions run serially—that is, one at a time, each starting only after the previous one has completed.

However, there are two good reasons for allowing concurrency

- Improved throughput and resource utilization.
- Reduced waiting time.

The database system must control the interaction among the concurrent transactions to prevent them from destroying the consistency of the database. It does so through a variety of mechanisms called concurrency-control schemes.

The execution sequences just described are called **schedules**. They represent the chronological order in which instructions are executed in the system

These schedules are serial: Each serial schedule consists of a sequence of instructions from various transactions, where the instructions belonging to one single transaction appear together in that schedule.

When the database system executes several transactions concurrently, the corresponding schedule no longer needs to be serial. If two transactions are running concurrently, the operating system may execute one transaction for a little while, then perform a context switch, execute the second transaction for sometime, and then switch back to the first transaction for some time, and so on. With multiple transactions, the CPU time is shared among all the transactions.

We can ensure consistency of the database under concurrent execution by making sure that any schedule that is executed has the same effect as a schedule that could have occurred without any concurrent execution. That is, the schedule should, in some sense, be equivalent to a serial schedule. Such schedules are called **serializable** schedules.

Screen clipping taken: 12-06-2026 08:05

Implementation of Isolation Levels

Screen clipping taken: 12-06-2026 08:17

There are various concurrency-control policies that we can use to ensure that, even when multiple transactions are executed concurrently, only acceptable schedules are generated, regardless of how the operating system time-shares resources (such as CPU time) among the transactions.

A transaction acquires a lock on the entire database before it starts and releases the **lock** after it has committed.

The goal of concurrency-control policies is to provide a high degree of concurrency, while ensuring that all schedules that can be generated are conflict or view serializable, recoverable, and cascadeless.

Locking

Instead of locking the entire database, a transaction could, instead, lock only those data items that it accesses. Under such a policy, the transaction must hold locks long enough to ensure serializability, we present the two-phase locking protocol, a simple, widely used technique that ensures serializability. Stated simply, two-phase locking requires a transaction to have two phases, one where it acquires locks but does not release any, A second phase where the transaction releases locks but does not acquire any.

Further improvements to locking result if we have two kinds of locks:

shared and exclusive.

Shared locks are used for data that the transaction **reads** and exclusive locks are used for those it **writes**.

Many transactions can hold shared locks on the same data item at the same time, a transaction is allowed an exclusive lock on a data item only if no other transaction holds any lock (regardless of whether shared or exclusive) on the data item.

Timestamps

Another category of techniques for the implementation of isolation assigns each transaction a **timestamp**, For each data item, the system keeps two timestamps.

1. read timestamp
2. write timestamp

The read timestamp of a data item holds the largest (that is, the most recent) timestamp of those transactions that read the data item.

The write timestamp of a data item holds the timestamp of the transaction that wrote the current value of the data item.

Timestamps are used to ensure that transactions access each data item in order of the transactions' time stamps if their accesses conflict.

Multiple Versions and Snapshot Isolation

By maintaining more than one version of a data item, it is possible to allow a transaction to read an old version of a data item rather than a newer version written by an uncommitted transaction or by a transaction that should come later in the serialization order.

There are a variety of multi version concurrency control techniques. One in particular, called **snapshot isolation**, is widely used in practice.

In snapshot isolation, we can imagine that each transaction is given its own version, or snapshot, of the database when it begins.

It reads data from this private version and is thus isolated from the updates made by other transactions.

If the transaction updates the database, that update appears only in its own version, not in the actual database itself.

Information about these updates is saved so that the updates can be applied to the "real" database if the transaction commits.

BDE Discussion

1. Intro + about event

2. Appreciate + reward

3. Show value

You can use notes 📝, reactions ❤️ and more to voice your thoughts

Rakesh Kurakula - InstaServe

SP TN S RK KP

Follow me Just now

Screen clipping taken: 12-06-2026 11:25

Goal setting

Use this template to figure out what goals you want for your project to be successful. For goals, consider what you hope to achieve. Use the SMART (Specific, Measurable, Attainable, Relevant, Time-Bound) framework to guide your thinking. In order to identify anti-goals, consider what you explicitly don't want to do. Anti-goals may be things that are not in scope or they may be things that are actually in conflict with your goals.


Strategy
Project planning

Linkedin

Whatsapp

Unknown User

1. New Connections
2. Introduction
3. Build connections from network

Unknown User

1. Intro + about event
2. Appreciate + reward
3. Show value

Unknown User

1. Appreciating their work in email
2. Intro about yourself
3. About the company + event
4. Followup - FOMO - Fear of Missing out
5. Final call

Rakesh Kurakula - InstaServe



21%



Screen clipping taken: 12-06-2026 11:26